

Jumble — Project Spec

Author: Nusrath Fathima **Last updated:** 17 September 2026 **Status:** Planning — spec complete, schema designed (see companion schema document)

1. Problem statement

Parents run out of ideas to keep kids engaged, especially when they want to avoid defaulting to a screen. Existing "kids activity list" sites are static content — they don't know your child, don't remember what you've already tried, and don't adapt when an idea flops. Jumble is a personalized activity recommender: it learns each child's preferences over time and filters on real constraints (time available, materials on hand, weather) instead of handing back a generic list.

Target user: a parent or caregiver of one or more children, roughly ages 2–12, who wants a quick, tailored suggestion rather than another scroll through Pinterest.

The specific promise: only suggest things you can actually do right now, with what you already have.

On the name. *Jumble* means a mixed assortment, which is true of the product twice over: the mix of activity types it draws from (craft, science, outdoor, active, imaginative — deliberately not one category), and the jumble of odds and ends in a household drawer that the matching engine works against. It is short, easily spoken, tied to no single activity type, and sits naturally beside *Mealody* in the wider portfolio.

2. Competitive landscape and positioning

The category is crowded, but the crowding is shallow. Existing offerings fall into two groups.

Static content. Blog posts, printable "boredom buster jar" lists, and apps that are essentially a browsable list with category tabs. These are the overwhelming majority. They have no per-child state, no memory of what's been tried, and no adaptation. A representative App Store example advertises 100+ screen-free activities but, by its own privacy disclosure, collects no data at all — meaning it cannot remember anything about a child between sessions.

Local event and class finders. Apps like Anjou and WonderKit, which surface paid classes, camps, and enrichment programs nearby. These are more sophisticated — Anjou has per-child profiles and a "fit score" ranking on age, distance, schedule, and budget — but they solve a different problem. They help you sign up for something in advance. They do not answer "it's raining, I have forty minutes before dinner, and I need something to do with a six-year-old using what's already in my kitchen."

The gap. No one in the free, at-home segment is doing stateful per-child learning combined with live constraint filtering. That is the space this project occupies.

Differentiators, in order of importance

1. **Materials on hand.** The single biggest reason activity lists fail parents is that half the suggestions require supplies they don't have. A parent sets up their supply inventory once; from then on, only activities they can actually complete are shown. This is also the most backend-interesting feature — an inventory model with set-intersection matching against each activity's requirements.
2. **Explain every suggestion.** Each recommendation shows its reasoning: "Emma has loved two other building activities, it's dry outside, you said thirty minutes, and you have everything for it." This is the frontend expression of the backend algorithm, and it is nearly free to build precisely because the scoring is transparent weighted matching rather than a black box.
3. **Mess level as a first-class filter.** "Low mess, it's a school night" is something parents genuinely want and no existing product exposes as a control.
4. **Honest time budgeting.** Activity blogs systematically understate duration by ignoring setup and cleanup. Storing realistic total time — cleanup included — is a small decision that makes the product feel trustworthy.
5. **Per-child rather than per-household learning.** Most apps treat "the kids" as one undifferentiated group. Because weights hang off the child record, two siblings in the same house receive genuinely different rankings.

3. MVP feature set

These ship first — enough to demonstrate the full loop end to end.

1. **Parent account** — sign up and log in.
2. **Child profiles** — a parent can add multiple children, each with a name, age, and interest tags (art, science, outdoor, building, pretend play, cooking, quiet/reading).
3. **Supply inventory** — the parent records what they have on hand, once, with easy editing later.
4. **Get a suggestion** — the parent picks a child, states today's context (time available, indoor/outdoor, mess tolerance), and receives a ranked shortlist of 3-5 activities.
5. **Explained recommendations** — each suggestion displays why it was chosen.
6. **Activity library** — a curated, seeded set of activities, each tagged with age range, category, indoor/outdoor, realistic total duration, materials required, and mess level.
7. **Feedback loop** — after trying a suggestion, the parent marks it loved / meh / skip. This adjusts that child's future ranking for the relevant tags — a content-based filter with

feedback-adjusted weights, not a black-box model, so every ranking decision remains explainable.

8. **History** — a log of what's been tried per child, used to avoid repeats and to surface simple stats ("14 activities this month, favourite category: crafts").
9. **Weather-aware filtering** — today's local weather is pulled so outdoor activities aren't suggested in the rain.

4. Stretch features (post-MVP)

- Weekly digest email ("5 things to try with Emma this week") — a scheduled job.
- Shareable "we did this" photo log per completed activity.
- Household sharing — two parents on one account and child set.
- Admin view for curating and expanding the activity library without a redeploy.

5. Data entities (high level — full schema is the next step)

Entity	Purpose
Parent	Account and auth identity.
Child	Belongs to a Parent; has age and interest tags.
Activity	Library item, tagged with age range, category, indoor/outdoor, duration, mess level.
Material	A supply item (e.g. glue, cardboard, food colouring).
ActivityMaterial	Which materials a given activity requires.
ParentInventory	Which materials a given parent has on hand.
Completion	A Child tried an Activity on a date, with a feedback rating.
ChildTagWeight	Derived score per (Child, tag) pair, updated on feedback; read by the ranking query.

6. Recommendation logic

1. Start from the full activity library.
2. Filter out anything outside the child's age range; anything that doesn't fit today's indoor/outdoor, time, and mess constraints; anything requiring materials the parent

doesn't have; and anything completed too recently.

3. Score each remaining activity by summing the child's current weight for each of its tags. A child who has loved several building activities carries a high building weight, so construction-style activities rise to the top.
4. Return the top 3–5 by score, each with the reasons that produced it.
5. When feedback arrives, nudge the weights for that activity's tags up or down for that child. *Loved* pushes harder than *meh*; *skip* pushes down.

This is intentionally simple — a weighted content-based filter, not a recommender-systems research project. But it is a real algorithm with traceable behaviour, which is the point both for the product (explainability is a feature) and for interviews.

7. API surface (REST, high level)

POST	/api/auth/signup	
POST	/api/auth/login	
GET	/api/children	
POST	/api/children	
GET	/api/children/{id}	
GET	/api/materials	
GET	/api/inventory	
PUT	/api/inventory	
GET	/api/activities	browse and search the library
GET	/api/children/{id}/suggestions	core recommendation endpoint; today's context passed as query params
POST	/api/children/{id}/completions	log a completion plus feedback rating
GET	/api/children/{id}/completions	history
GET	/api/children/{id}/stats	aggregate stats

8. Non-functional decisions

Auth. Email and password, with BCrypt-hashed passwords and session or JWT-based auth. Exact choice settled at backend scaffolding.

Weather integration. A free provider such as Open-Meteo, which requires no API key, called once per parent location per day and cached server-side — the same cost-discipline

instinct applied to Mealody's AI calls, redirected here from token budgeting to API-call budgeting.

Scheduled jobs. Spring's `@Scheduled` for the weekly digest.

Database. PostgreSQL. The domain is genuinely relational — parent to child to completion to tag weight, plus two many-to-many joins for materials — so a relational store is the correct fit rather than a default.

9. Tech stack

Layer	Choice
Backend	Java + Spring Boot (Spring Web, Spring Data JPA, Spring Security, Spring Scheduler)
Database	PostgreSQL
Frontend	React
API and DB hosting	Render or Railway
Frontend hosting	Vercel

Spring Boot is a deliberate choice: it puts the Oracle Java certification to work, and nothing else in the portfolio currently does. Vercel's serverless-function model does not suit a stateful Spring application, hence the split hosting.

10. Interview talking points this project is designed to produce

- "I chose transparent weighted scoring over a black-box model because I could reason about and debug every recommendation — and then I turned that transparency into a user-facing feature that explains each suggestion."
 - "I cached weather lookups per location per day to avoid unnecessary external API calls — the same cost-discipline principle I applied to AI usage on a previous project, applied to a different resource."
 - "I modelled a genuinely relational domain in Spring Data JPA, including a derived weights table maintained on every feedback event, and made a deliberate read-performance-versus-write-complexity tradeoff to keep the hot path fast."
 - "This is my first production Spring Boot build, which puts my Oracle Java certification into practice in a full-stack context."
-

11. Open questions for the next step (schema design)

1. Store `ChildTagWeight` as its own table (fast reads, upkeep on every feedback event) or compute it on the fly from completion history (simpler writes, heavier reads)? *Leaning: dedicated table — the suggestions endpoint is the hot path.*
2. How many days should exclude a repeat suggestion? *Leaning: configurable, default 14.*
3. Household sharing — MVP or stretch? *Leaning: stretch; one parent owns each child for now, since adding a join table later is a clean additive migration.*